REFERENCE

Channels reference

Build an MCP server that pushes webhooks, alerts, and chat messages into a Claude Code session. Reference for the channel contract: capability declaration, notification events, reply tools, sender gating, and permission relay.

Channels are in **research preview** and require Claude Code v2.1.80 or later. Team and Enterprise organizations must **explicitly enable them**.

A channel is an MCP server that pushes events into a Claude Code session so Claude can react to things happening outside the terminal.

You can build a one-way or two-way channel. One-way channels forward alerts, webhooks, or monitoring events for Claude to act on. Two-way channels like chat bridges also **expose a reply tool** so Claude can send messages back. A channel with a trusted sender path can also opt in to **relay permission prompts** so you can approve or deny tool use remotely.

This page covers:

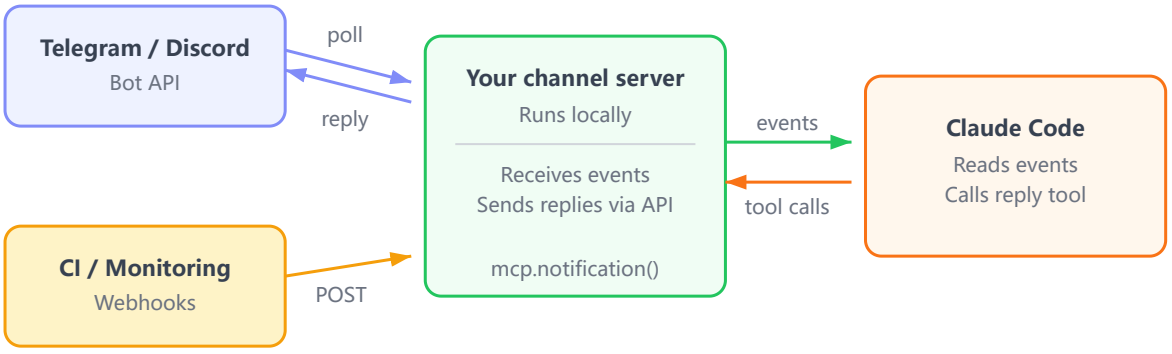
- **Overview**: how channels work
- **What you need**: requirements and general steps
- **Example: build a webhook receiver**: a minimal one-way walkthrough
- **Server options**: the constructor fields
- **Notification format**: the event payload and delivery behavior
- **Expose a reply tool**: let Claude send messages back
- **Gate inbound messages**: sender checks to prevent prompt injection
- **Relay permission prompts**: forward tool approval prompts to remote channels

To use an existing channel instead of building one, see **Channels**. Telegram, Discord, iMessage, and fakechat are included in the research preview.

Overview

A channel is an **MCP** server that runs on the same machine as Claude Code. Claude Code spawns it as a subprocess and communicates over stdio. Your channel server is the bridge between external systems and the Claude Code session:

- **Chat platforms** (Telegram, Discord): your plugin runs locally and polls the platform's API for new messages. When someone DMs your bot, the plugin receives the message and forwards it to Claude. No URL to expose.
- **Webhooks** (CI, monitoring): your server listens on a local HTTP port. External systems POST to that port, and your server pushes the payload to Claude.



What you need

The only hard requirement is the `@modelcontextprotocol/sdk` package and a Node.js-compatible runtime. **Bun**, **Node**, and **Deno** all work. The pre-built plugins in the research preview use Bun, but your channel doesn't have to.

Your server needs to:

1. Declare the `claude/channel` capability so Claude Code registers a notification listener
2. Emit `notifications/claude/channel` events when something happens
3. Connect over **stdio transport** (Claude Code spawns your server as a subprocess)

The **Server options** and **Notification format** sections cover each of these in detail. See **Example: build a webhook receiver** for a full walkthrough.

During the research preview, custom channels aren't on the **approved allowlist**. Use `--dangerously-load-development-channels` to test locally. See **Test during the research preview** for details.

Example: build a webhook receiver

This walkthrough builds a single-file server that listens for HTTP requests and forwards them into your Claude Code session. By the end, anything that can send an HTTP POST, like a CI pipeline, a monitoring alert, or a `curl` command, can push events to Claude.

This example uses **Bun** as the runtime for its built-in HTTP server and TypeScript support. You can use **Node** or **Deno** instead; the only requirement is the **MCP SDK**.

1 Create the project

Create a new directory and install the MCP SDK:

```
mkdir webhook-channel && cd webhook-channel
bun add @modelcontextprotocol/sdk
```

2 Write the channel server

Create a file called `webhook.ts`. This is your entire channel server: it connects to Claude Code over stdio, and it listens for HTTP POSTs on port 8788. When a request arrives, it pushes the body to Claude as a channel event.

webhook.ts

```ts
#!/usr/bin/env bun
import { Server } from '@modelcontextprotocol/sdk/server/index.js'
import { StdioServerTransport } from
'@modelcontextprotocol/sdk/server/stdio.js'

// Create the MCP server and declare it as a channel
const mcp = new Server(
  { name: 'webhook', version: '0.0.1' },
  {
    // this key is what makes it a channel — Claude Code registers a
listener for it
    capabilities: { experimental: { 'claude/channel': {} } },
    // added to Claude's system prompt so it knows how to handle these
events
    instructions: 'Events from the webhook channel arrive as <channel
source="webhook" ...>. They are one-way: read them and act, no reply
expected.',
  },
)

// Connect to Claude Code over stdio (Claude Code spawns this process)
await mcp.connect(new StdioServerTransport())

// Start an HTTP server that forwards every POST to Claude
Bun.serve({
  port: 8788,  // any open port works
  // localhost-only: nothing outside this machine can POST
  hostname: '127.0.0.1',
  async fetch(req) {
    const body = await req.text()
    await mcp.notification({
      method: 'notifications/claude/channel',
      params: {
        content: body,  // becomes the body of the <channel> tag
        // each key becomes a tag attribute, e.g. <channel path="/"
method="POST">
        meta: { path: new URL(req.url).pathname, method: req.method },
      },
    })
    return new Response('ok')
  },
})
```

The file does three things in order:

- **Server configuration**: creates the MCP server with `claude/channel` in its capabilities, which is what tells Claude Code this is a channel. The `instructions` string goes into Claude's system prompt: tell Claude what events to expect, whether to reply, and how to route replies if it should.
- **Stdio connection**: connects to Claude Code over stdin/stdout. This is standard for any **MCP server**: Claude Code spawns it as a subprocess.
- **HTTP listener**: starts a local web server on port 8788. Every POST body gets forwarded to Claude as a channel event via `mcp.notification()`. The `content` becomes the event body, and each `meta` entry becomes an attribute on the `<channel>` tag. The listener needs access to the `mcp` instance, so it runs in the same process. You could split it into separate modules for a larger project.

3 Register your server with Claude Code

Add the server to your MCP config so Claude Code knows how to start it. For a project-level `.mcp.json` in the same directory, use a relative path. For user-level config in `~/.claude.json`, use the full absolute path so the server can be found from any project:

.mcp.json

```
{
  "mcpServers": {
    "webhook": { "command": "bun", "args": ["./webhook.ts"] }
  }
}
```

Claude Code reads your MCP config at startup and spawns each server as a subprocess.

4 Test it

During the research preview, custom channels aren't on the allowlist, so start Claude Code with the development flag:

```
claude --dangerously-load-development-channels
server:webhook
```

The first time you start a session in this project, Claude Code asks for consent before using the new server from `.mcp.json`. The dialog reports "New MCP server found in this project: webhook". Select **Use this MCP server** to continue.

When Claude Code starts, it reads your MCP config, spawns your `webhook.ts` as a subprocess, and the HTTP listener starts automatically on the port you configured (8788 in this example). You don't need to run the server yourself.

A dim notice below the startup banner confirms the channel is registered: `Channels (experimental) messages from server:webhook inject directly in this session · restart without --dangerously-load-development-channels to stop`.

If you see "blocked by org policy," your organization admin needs to **enable channels** first.

In a separate terminal, simulate a webhook by sending an HTTP POST with a message to your server. This example sends a CI failure alert to port 8788 (or whichever port you configured):

```
curl -X POST localhost:8788 -d "build failed on main:
https://ci.example.com/run/1234"
```

The payload arrives in your Claude Code session as a `<channel>` tag:

```
<channel source="webhook" path="/" method="POST">build
failed on main: https://ci.example.com/run/1234</channel>
```

In your Claude Code terminal, you'll see Claude receive the message and start responding: reading files, running commands, or whatever the message calls for. This is a one-way channel, so Claude acts in your session but doesn't send anything back through the webhook. To add replies, see **Expose a reply tool**.

If the event doesn't arrive, the diagnosis depends on what `curl` returned:

- `curl` **succeeds but nothing reaches Claude**: run `/mcp` in your session to check the server's status. "Failed to connect" usually means a dependency or import error in your server file; check the debug log at `~/.claude/debug/<session-id>.txt` for the stderr trace.
- `curl` **fails with "connection refused"**: the port is either not bound yet or a stale process from an earlier run is holding it. `lsof -i :<port>` shows what's listening; `kill` the stale

process before restarting your session.

The **fakechat server** extends this pattern with a web UI, file attachments, and a reply tool for two-way chat.

Test during the research preview

During the research preview, every channel must be on the **approved allowlist** to register. The development flag bypasses the allowlist for specific entries after a confirmation prompt. This example shows both entry types:

```
# Testing a plugin you're developing
claude --dangerously-load-development-channels
plugin:yourplugin@yourmarketplace

# Testing a bare .mcp.json server (no plugin wrapper yet)
claude --dangerously-load-development-channels server:webhook
```

The bypass is per-entry. Combining this flag with `--channels` doesn't extend the bypass to the `--channels` entries. During the research preview, the approved allowlist is Anthropic-curated, so your channel stays on the development flag while you build and test.

> This flag skips the allowlist only. The `channelsEnabled` organization policy still applies. Don't use it to run channels from untrusted sources.

Server options

A channel sets these options in the `Server` constructor. The `instructions` and `capabilities.tools` fields are **standard MCP**; `capabilities.experimental['claude/channel']` and `capabilities.experimental['claude/channel/permission']` are the channel-specific additions:

Field	Type	Description
`capabilities.experimental['claude/channel']`	`object`	Required. Always `{}`. Presence registers the notification listener.
`capabilities.experimental`	`object`	Optional. Always `{}`. Declares that this channel can

Field	Type	Description
`['claude/channel/permission']`		receive permission relay requests. When declared, Claude Code forwards tool approval prompts to your channel so you can approve or deny them remotely. See **Relay permission prompts**.
`capabilities.tools`	`object`	Two-way only. Always `{}`. Standard MCP tool capability. See **Expose a reply tool**.
`instructions`	`string`	Recommended. Added to Claude's system prompt. Tell Claude what events to expect, what the `<channel>` tag attributes mean, whether to reply, and if so which tool to use and which attribute to pass back (like `chat_id`).

To create a one-way channel, omit `capabilities.tools`. This example shows a two-way setup with the channel capability, tools, and instructions set:

```
import { Server } from
'@modelcontextprotocol/sdk/server/index.js'

const mcp = new Server(
  { name: 'your-channel', version: '0.0.1' },
  {
    capabilities: {
      experimental: { 'claude/channel': {} },  // registers the
channel listener
      tools: {},  // omit for one-way channels
    },
    // added to Claude's system prompt so it knows how to handle
your events
    instructions: 'Messages arrive as <channel source="your-
channel" ...>. Reply with the reply tool.',
  },
)
```

To push an event, call `mcp.notification()` with method `notifications/claude/channel`. The params are in the next section.

Notification format

Your server emits `notifications/claude/channel` with two params:

Field	Type	Description
`content`	`string`	The event body. Delivered as the body of the `<channel>` tag.
`meta`	`Record<string, string>`	Optional. Each entry becomes an attribute on the `<channel>` tag for routing context like chat ID, sender name, or alert severity. Keys must be identifiers: letters, digits, and underscores only. Keys containing hyphens or other characters are silently dropped.

Your server pushes events by calling `mcp.notification()` on the `Server` instance. This example pushes a CI failure alert with two meta keys:

```
await mcp.notification({
  method: 'notifications/claude/channel',
  params: {
    content: 'build failed on main:
https://ci.example.com/run/1234',
    meta: { severity: 'high', run_id: '1234' },
  },
})
```

The event arrives in Claude's context wrapped in a `<channel>` tag. The `source` attribute is set automatically from your server's configured name:

```
<channel source="your-channel" severity="high" run_id="1234">
build failed on main: https://ci.example.com/run/1234
</channel>
```

Notifications are not acknowledged. The `await` on `mcp.notification()` resolves when the message is written to the transport, not when Claude has processed it. If the session hasn't loaded your server as a channel, or the organization policy blocks it, events are dropped silently with no error returned to your server.

If you need delivery confirmation, track event state in your server and expose a **reply tool** that Claude can call to report status back.

Events queue into the session and are processed in order. If several notifications arrive while Claude is busy, they're delivered together on the next turn and Claude handles them as a group. To process independent event streams concurrently, run separate sessions.

Expose a reply tool

If your channel is two-way, like a chat bridge rather than an alert forwarder, expose a standard **MCP tool** that Claude can call to send messages back. Nothing about the tool registration is channel-specific. A reply tool has three components:

1. A `tools: {}` entry in your `Server` constructor capabilities so Claude Code discovers the tool
2. Tool handlers that define the tool's schema and implement the send logic
3. An `instructions` string in your `Server` constructor that tells Claude when and how to call the tool

To add these to the **webhook receiver above**:

1 Enable tool discovery

In your `Server` constructor in `webhook.ts`, add `tools: {}` to the capabilities so Claude Code knows your server offers tools:

```
capabilities: {
  experimental: { 'claude/channel': {} },
  tools: {},  // enables tool discovery
},
```

2 Register the reply tool

Add the following to `webhook.ts`. The `import` goes at the top of the file with your other imports; the two handlers go between the `Server` constructor and `mcp.connect()`. This registers a `reply` tool that Claude can call with a `chat_id` and `text`:

```
// Add this import at the top of webhook.ts
import { ListToolsRequestSchema, CallToolRequestSchema }
from '@modelcontextprotocol/sdk/types.js'

// Claude queries this at startup to discover what tools
your server offers
mcp.setRequestHandler(ListToolsRequestSchema, async () => ({
  tools: [{
    name: 'reply',
    description: 'Send a message back over this channel',
    // inputSchema tells Claude what arguments to pass
    inputSchema: {
      type: 'object',
      properties: {
        chat_id: { type: 'string', description: 'The
conversation to reply in' },
        text: { type: 'string', description: 'The message to
send' },
      },
      required: ['chat_id', 'text'],
    },
  }],
}))

// Claude calls this when it wants to invoke a tool
mcp.setRequestHandler(CallToolRequestSchema, async req => {
  if (req.params.name === 'reply') {
    const { chat_id, text } = req.params.arguments as {
chat_id: string; text: string }
    // send() is your outbound: POST to your chat platform,
or for local
    // testing the SSE broadcast shown in the full example
below.
    send(`Reply to ${chat_id}: ${text}`)
    return { content: [{ type: 'text', text: 'sent' }] }
  }
  throw new Error(`unknown tool: ${req.params.name}`)
})
```

3 Update the instructions

Update the `instructions` string in your `Server` constructor so Claude knows to route

replies back through the tool. This example tells Claude to pass `chat_id` from the inbound tag:

```
instructions: 'Messages arrive as <channel source="webhook"
chat_id="...">. Reply with the reply tool, passing the
chat_id from the tag.'
```

Here's the complete `webhook.ts` with two-way support. Outbound replies stream over `GET /events` using **Server-Sent Events** (SSE), so `curl -N localhost:8788/events` can watch them live; inbound chat arrives on `POST /`:

Full webhook.ts with reply tool

```
#!/usr/bin/env bun
import { Server } from '@modelcontextprotocol/sdk/server/index.js'
import { StdioServerTransport } from
'@modelcontextprotocol/sdk/server/stdio.js'
import { ListToolsRequestSchema, CallToolRequestSchema } from
'@modelcontextprotocol/sdk/types.js'
```

··· See all 86 lines

The **fakechat server** shows a more complete example with file attachments and message editing.

Gate inbound messages

An ungated channel is a prompt injection vector. Anyone who can reach your endpoint can put text in front of Claude. A channel listening to a chat platform or a public endpoint needs a real sender check before it emits anything.

Check the sender against an allowlist before calling `mcp.notification()`. This example drops any message from a sender not in the set:

```
const allowed = new Set(loadAllowlist())  // from your
access.json or equivalent

// inside your message handler, before emitting:
if (!allowed.has(message.from.id)) {  // sender, not room
  return  // drop silently
}
await mcp.notification({ ... })
```

Gate on the sender's identity, not the chat or room identity: `message.from.id` in the example, not `message.chat.id`. In group chats, these differ, and gating on the room would let anyone in an allowlisted group inject messages into the session.

The **Telegram** and **Discord** channels gate on a sender allowlist the same way. They bootstrap the list by pairing: the user DMs the bot, the bot replies with a pairing code, the user approves it in their Claude Code session, and their platform ID is added. See either implementation for the full pairing flow. The **iMessage** channel takes a different approach: it detects the user's own addresses from the Messages database at startup and lets them through automatically, with other senders added by handle.

Relay permission prompts

> Permission relay requires Claude Code v2.1.81 or later. Earlier versions ignore the `claude/channel/permission` capability.

When Claude calls a tool that needs approval, the local terminal dialog opens and the session waits. A two-way channel can opt in to receive the same prompt in parallel and relay it to you on another device. Both stay live: you can answer in the terminal or on your phone, and Claude Code applies whichever answer arrives first and closes the other.

Relay covers tool-use approvals like `Bash`, `Write`, and `Edit`. Project trust and MCP server consent dialogs don't relay; those only appear in the local terminal.

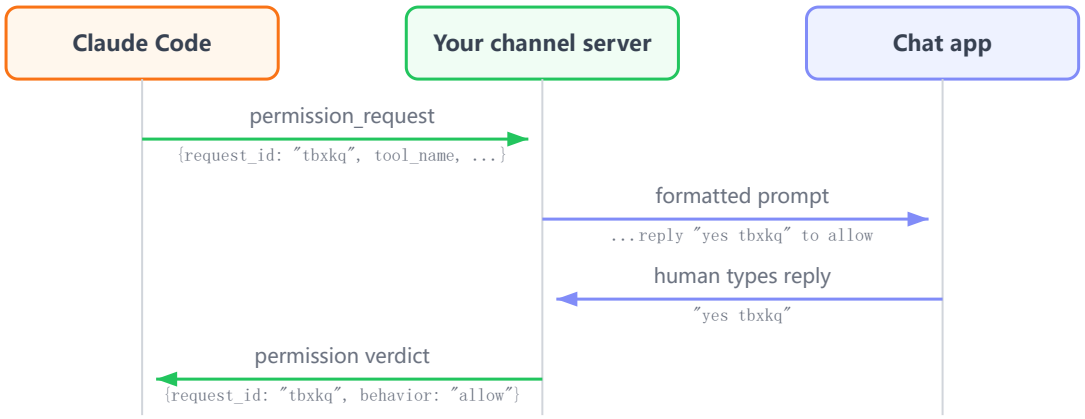
How relay works

When a permission prompt opens, the relay loop has four steps:

1. Claude Code generates a short request ID and notifies your server

2. Your server forwards the prompt and ID to your chat app
3. The remote user replies with a yes or no and that ID
4. Your inbound handler parses the reply into a verdict, and Claude Code applies it only if the ID matches an open request

The local terminal dialog stays open through all of this. If someone at the terminal answers before the remote verdict arrives, that answer is applied instead and the pending remote request is dropped.



Permission request fields

The outbound notification from Claude Code is `notifications/claude/channel/permission_request`. Like the **channel notification**, the transport is standard MCP but the method and schema are Claude Code extensions. The `params` object has four string fields your server formats into the outgoing prompt:

Field	Description
`request_id`	Five lowercase letters drawn from `a` - `z` without `l`, so it never reads as a `1` or `I` when typed on a phone. Include it in your outgoing prompt so it can be echoed in the reply. Claude Code only accepts a verdict that carries an ID it issued. The local terminal dialog doesn't display this ID, so your outbound handler is the only way to learn it.
`tool_name`	Name of the tool Claude wants to use, for example `Bash` or `Write`.
`description`	Human-readable summary of what this specific tool call does, the same text the local terminal dialog shows. For a Bash call this is Claude's description of the command, or the command itself if none was given.
`input_preview`	The tool's arguments as a JSON string, truncated to 200 characters. For Bash this is the command; for Write it's the file path and a prefix of the content. Omit it from your prompt if you only have room for a one-line message. Your server decides what to show.

The verdict your server sends back is `notifications/claude/channel/permission` with two fields:

`request_id` echoing the ID above, and `behavior` set to `'allow'` or `'deny'`. Allow lets the tool call proceed; deny rejects it, the same as answering No in the local dialog. Neither verdict affects future calls.

Add relay to a chat bridge

Adding permission relay to a two-way channel takes three components:

1. A `claude/channel/permission: {}` entry under `experimental` capabilities in your `Server` constructor so Claude Code knows to forward prompts
2. A notification handler for `notifications/claude/channel/permission_request` that formats the prompt and sends it out through your platform API
3. A check in your inbound message handler that recognizes `yes <id>` or `no <id>` and emits a `notifications/claude/channel/permission` verdict instead of forwarding the text to Claude

Only declare the capability if your channel **authenticates the sender**, because anyone who can reply through your channel can approve or deny tool use in your session.

To add these to a two-way chat bridge like the one assembled in **Expose a reply tool**:

1 Declare the permission capability

In your `Server` constructor, add `claude/channel/permission: {}` alongside `claude/channel` under `experimental`:

```
capabilities: {
  experimental: {
    'claude/channel': {},
    'claude/channel/permission': {},  // opt in to
permission relay
  },
  tools: {},
},
```

2 Handle the incoming request

Register a notification handler between your `Server` constructor and `mcp.connect()`. Claude Code calls it with the **four request fields** when a permission dialog opens. Your handler

formats the prompt for your platform and includes instructions for replying with the ID:

```
import { z } from 'zod'

// setNotificationHandler routes by z.literal on the method
field,
// so this schema is both the validator and the dispatch key
const PermissionRequestSchema = z.object({
  method:
z.literal('notifications/claude/channel/permission_request')
,
  params: z.object({
    request_id: z.string(),      // five lowercase letters,
include verbatim in your prompt
    tool_name: z.string(),       // e.g. "Bash", "Write"
    description: z.string(),     // human-readable summary of
this call
    input_preview: z.string(),   // tool args as JSON,
truncated to ~200 chars
  }),
})

mcp.setNotificationHandler(PermissionRequestSchema, async ({
params }) => {
  // send() is your outbound: POST to your chat platform, or
for local
  // testing the SSE broadcast shown in the full example
below.
  send(
    `Claude wants to run ${params.tool_name}:
${params.description}\n\n` +
    // the ID in the instruction is what your inbound
handler parses in Step 3
    `Reply "yes ${params.request_id}" or "no
${params.request_id}"`,
  )
})
```

3 Intercept the verdict in your inbound handler

Your inbound handler is the loop or callback that receives messages from your platform: the same place you **gate on sender** and emit `notifications/claude/channel` to forward chat to

Claude. Add a check before the chat-forwarding call that recognizes the verdict format and emits the permission notification instead.

The regex matches the ID format Claude Code generates: five letters, never `l`. The `/i` flag tolerates phone autocorrect capitalizing the reply; lowercase the captured ID before sending it back.

```
// matches "y abcde", "yes abcde", "n abcde", "no abcde"
// [a-km-z] is the ID alphabet Claude Code uses (lowercase,
skips 'l')
// /i tolerates phone autocorrect; lowercase the capture
before sending
const PERMISSION_REPLY_RE = /^\s*(y|yes|n|no)\s+([a-km-z]
{5})\s*$/i

async function onInbound(message: PlatformMessage) {
  if (!allowed.has(message.from.id)) return  // gate on
sender first

  const m = PERMISSION_REPLY_RE.exec(message.text)
  if (m) {
    // m[1] is the verdict word, m[2] is the request ID
    // emit the verdict notification back to Claude Code
instead of chat
    await mcp.notification({
      method: 'notifications/claude/channel/permission',
      params: {
        request_id: m[2].toLowerCase(),  // normalize in
case of autocorrect caps
        behavior: m[1].toLowerCase().startsWith('y') ?
'allow' : 'deny',
      },
    })
    return  // handled as verdict, don't also forward as
chat
  }

  // didn't match verdict format: fall through to the normal
chat path
  await mcp.notification({
    method: 'notifications/claude/channel',
    params: { content: message.text, meta: { chat_id:
String(message.chat.id) } },
  })
}
```

Claude Code also keeps the local terminal dialog open, so you can answer in either place, and the first answer to arrive is applied. A remote reply that doesn't exactly match the expected format fails

in one of two ways, and in both cases the dialog stays open:

- **Different format**: your inbound handler's regex fails to match, so text like `approve it` or `yes` without an ID falls through as a normal message to Claude.
- **Right format, wrong ID**: your server emits a verdict, but Claude Code finds no open request with that ID and drops it silently.

Full example

The assembled `webhook.ts` below combines all three extensions from this page: the reply tool, sender gating, and permission relay. If you're starting here, you'll also need the **project setup and `.mcp.json` entry** from the initial walkthrough.

To make both directions testable from curl, the HTTP listener serves two paths:

- `GET /events`: holds an SSE stream open and pushes each outbound message as a `data:` line, so `curl -N` can watch Claude's replies and permission prompts arrive live.
- `POST /`: the inbound side, the same handler as earlier, now with the verdict-format check inserted before the chat-forward branch.

Full webhook.ts with permission relay

```
#!/usr/bin/env bun
import { Server } from '@modelcontextprotocol/sdk/server/index.js'
import { StdioServerTransport } from
'@modelcontextprotocol/sdk/server/stdio.js'
import { ListToolsRequestSchema, CallToolRequestSchema } from
'@modelcontextprotocol/sdk/types.js'
import { z } from 'zod'
```

··· See all 132 lines

Test the verdict path in three terminals. The first is your Claude Code session, started with the **development flag** so it spawns `webhook.ts`:

```
claude --dangerously-load-development-channels server:webhook
```

In the second, stream the outbound side so you can see Claude's replies and any permission prompts as they fire:

```
curl -N localhost:8788/events
```

In the third, send a message that will make Claude try to run a command:

```
curl -d "list the files in this directory" -H "X-Sender: dev"
localhost:8788
```

Listing files is read-only, so Claude runs it without approval. The permission dialog opens when Claude calls the `reply` tool to send its answer back. The local dialog opens in your Claude Code terminal, and a moment later the prompt for `mcp__webhook__reply` appears in the `/events` stream, including the five-letter ID. Approve it from the remote side:

```
curl -d "yes <id>" -H "X-Sender: dev" localhost:8788
```

The local dialog closes, the `reply` tool runs, and Claude's reply lands in the stream.

The three channel-specific pieces in this file:

- **Capabilities** in the `Server` constructor: `claude/channel` registers the notification listener, `claude/channel/permission` opts in to permission relay, `tools` lets Claude discover the reply tool.
- **Outbound paths**: the `reply` tool handler is what Claude calls for conversational responses; the `PermissionRequestSchema` notification handler is what Claude Code calls when a permission dialog opens. Both call `send()` to broadcast over `/events`, but they're triggered by different parts of the system.
- **HTTP handler**: `GET /events` holds an SSE stream open so curl can watch outbound live; `POST` is inbound, gated on the `X-Sender` header. A `yes <id>` or `no <id>` body goes to Claude Code as a verdict notification and never reaches Claude; anything else is forwarded to Claude as a channel event.

Package as a plugin

To make your channel installable and shareable, wrap it in a **plugin** and publish it to a **marketplace**. Users install it with `/plugin install`, then enable it per session with `--channels plugin:<name>@<marketplace>`.

A channel published to your own marketplace still needs `--dangerously-load-development-channels` to run, since it isn't on the **approved allowlist**. The default allowlist is the channel plugins in `claude-plugins-official`, which Anthropic curates at its discretion. The **in-app submission forms** add plugins to the community marketplace, which is not on the channel allowlist.

If you are working with an Anthropic partner contact, reach out to them to coordinate an official-marketplace listing. On Team and Enterprise plans, an admin can instead include your plugin in the organization's own `allowedChannelPlugins` list, which replaces the default Anthropic allowlist.

See also

- **Channels** to install and use Telegram, Discord, iMessage, or the fakechat demo, and to enable channels for a Team or Enterprise org
- **Working channel implementations** for complete server code with pairing flows, reply tools, and file attachments
- **MCP** for the underlying protocol that channel servers implement
- **Plugins** to package your channel so users can install it with `/plugin install`